

AI-Powered Eye Tracking for OS Control

Comprehensive Guide & Roadmap

Date: 2026-04-19

1) Project Overview

Objective

Develop an accurate, robust, and real-time eye-tracking system capable of replacing a standard computer mouse. The system will map user gaze to screen coordinates (X, Y) and detect intentional blinks as click commands.

Target Scope

Semester Project — High complexity, AI-driven, combining:

- Computer Vision
- Machine Learning
- Human-Computer Interaction (HCI)
- Real-time OS-level interaction

2) Significant Contribution (Novelty)

Hybrid Landmark + AI Pipeline

Instead of computationally expensive CNN inference on raw images, the system uses MediaPipe for fast 3D facial landmark extraction and feeds these numerical features into a lightweight custom model.

Intentional Blink Detection

A time-series classification approach (e.g., SVM or lightweight RNN) distinguishes involuntary short blinks (~150 ms) from intentional click blinks (~400 ms).

Advanced Cursor Jitter Reduction

Real-time smoothing via Exponential Moving Average (EMA) or One Euro Filter ensures natural cursor behaviour and minimises erratic movement.

3) Step-by-Step Implementation Roadmap

Phase 1 — Core Detection Foundation

Goal: Reliable real-time tracking of face and eyes.

Tools: Python, OpenCV, MediaPipe (face_landmarker.task)

- Extract 468 3D facial landmarks per frame.
- Isolate landmark indices for: Left eye, Right eye, Iris regions.

Phase 2 — Custom Data Collection

Goal: Build personalised training datasets.

- **A) Gaze Dataset (5,000–15,000 samples):** Show a moving red dot; user follows it. Record iris 3D coordinates + head pose (Pitch, Yaw, Roll) as features; dot position (X, Y) as label.
- **B) Blink Dataset (500–1,000 samples):** Record EAR-based temporal sequences. Labels: 0 = open/natural blink, 1 = intentional click blink.

Phase 3 — AI Model Training

Goal: Train lightweight and deployable models.

- **Gaze Model:** Train an MLP or Random Forest to predict screen (X, Y) from eye/head features. Export as .keras (training) and .tflite (optimised inference).
- **Click Model:** Train a temporal classifier for intentional blink detection using blink-duration dynamics.

Phase 4 — Real-Time Calibration System

Goal: Adapt predictions to user posture and monitor geometry.

- Build a 5-second startup calibration UI.
- Collect calibration points at 4 corners + 1 centre point.
- Use Polynomial Regression to map model output to exact monitor coordinates.

Phase 5 — OS Integration & Smoothing

Goal: Enable practical full-system mouse control.

Tools: pyautogui / pynput

- Use pyautogui or pynput for cursor control and clicking.
- Apply smoothing on predicted (X, Y) trajectory via EMA and/or One Euro Filter to reduce jitter while preserving responsiveness.

4) Major Hurdles and Mitigation Strategies

Hurdle 1 — "Midas Touch" (False Clicks)

Problem: Normal blinks accidentally trigger clicks, making the system unusable.

Mitigation:

- Avoid static closed-eye thresholding.
- Model blink duration and click only for ~400–600 ms closures.
- Optional fallback: Dwell Clicking (e.g., 1-second gaze hold).

Hurdle 2 — Head Movement Drift

Problem: Small head shifts break gaze mapping.

Mitigation:

- Estimate head pose (Yaw, Pitch, Roll) from MediaPipe landmarks.
- Include pose features in gaze model so it learns eye-vs-head compensation.

Hurdle 3 — Cursor Jitter

Problem: Frame-level landmark noise causes unstable cursor movement.

Mitigation:

- Never drive cursor from raw per-frame output.
- Buffer recent predictions (5–10 frames).
- Apply One Euro Filter (preferred for HCI) or EMA.

Hurdle 4 — Lighting Variability

Problem: Illumination changes affect iris/landmark reliability.

Mitigation:

- Collect training data under: bright frontal lighting, dim room lighting, side lighting / partial shadows.
- This improves generalisation and robustness.